Envío de Evidencias MVP: PichangaBook Arequipa

Avance de Backend, Frontend, Diagramas y Planificación

Formación de Empresas de Base Tecnológica I — Universidad Católica San Pablo

Rodrigo Sierra Samuel Iman

5 de noviembre de 2026

1 Resumen del Avance del MVP

En este informe presentamos las evidencias del Producto Mínimo Viable (MVP) de **PichangaBook Arequipa** (repositorio `github.com/RodrigoSierr/PichangaBook`). El proyecto resuelve tres problemas frecuentes en el alquiler de canchas sintéticas en Arequipa: las reservas duplicadas entre recepción y WhatsApp, la dificultad del organizador para cobrar las cuotas individuales de sus compañeros y la falta de suplentes verificados de último minuto. El monorepo cuenta con tres carpetas conectadas entre sí: `/backend-nestjs` (API REST y WebSockets en NestJS + PostgreSQL), `/web-admin-tailwind` (panel web omnicanal para dueños) y `/mobile-flutter` (pantallas en Flutter/Dart y cliente móvil ejecutable).

2 Diagramas del Sistema

2.1 Diagrama de Arquitectura Backend–Frontend

El diagrama muestra los módulos reales configurados en `src/app.module.ts` y su comunicación con el panel web, la app móvil y la base de datos PostgreSQL:

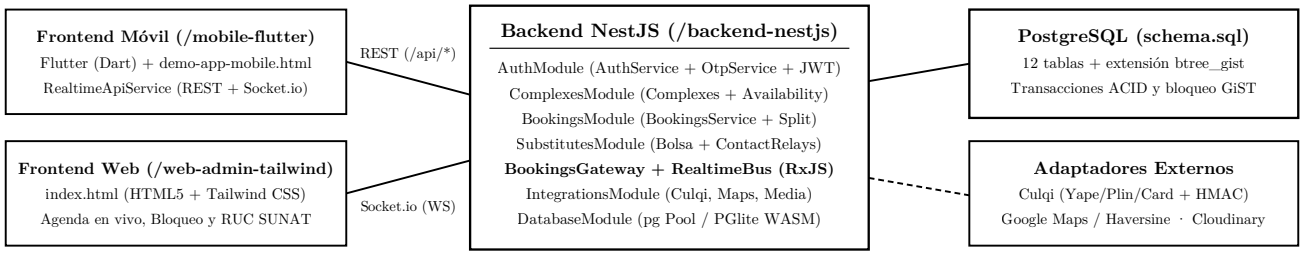


Figura 1: Diagrama de arquitectura del monorepo con los módulos de `AppModule` en NestJS.

2.2 Diagrama Entidad-Relación de la Base de Datos (PostgreSQL)

El siguiente diagrama Entidad-Relación refleja las **12 tablas** definidas en `backend-nestjs/schema.sql` con sus llaves primarias (`PK`), foráneas (`FK`) y restricciones de unicidad (`UQ`):

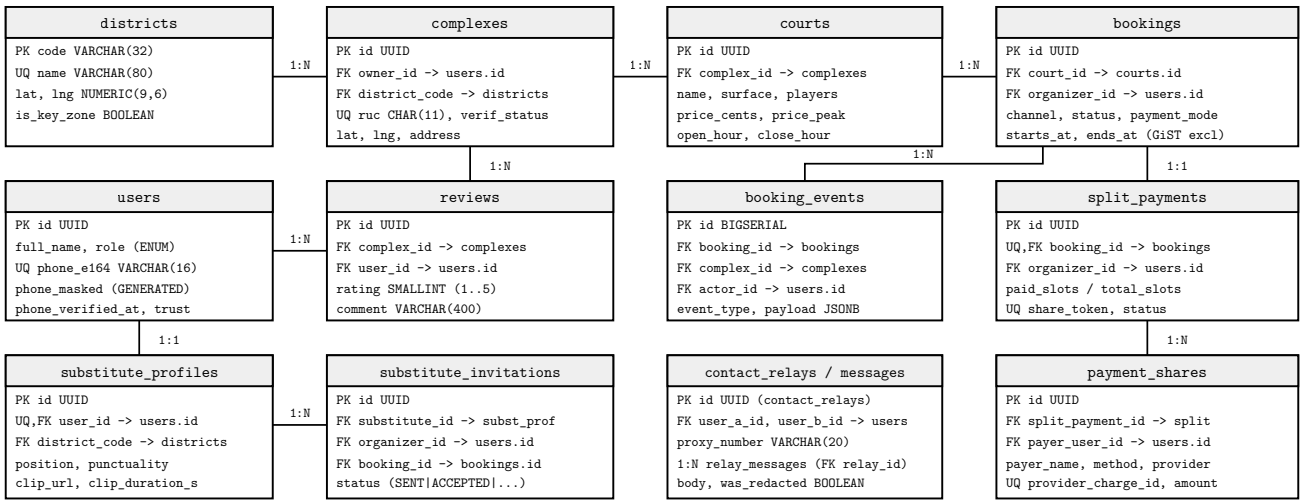


Figura 2: Diagrama Entidad-Relación completo de las 12 tablas definidas en `backend-nestjs/schema.sql`.

3 Evidencias de Código (Backend, Base de Datos y Frontend)

3.1 Esquema en PostgreSQL y restricción anti-doble reserva (`schema.sql`)

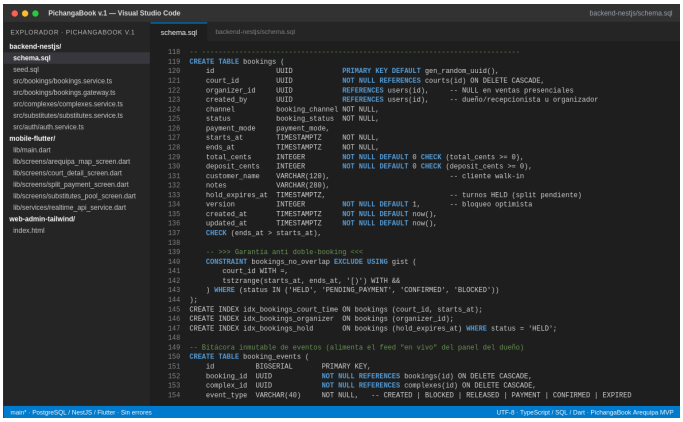


Figura 3: Código SQL de la tabla `bookings` y su restricción GiST en `backend-nestjs/schema.sql`.

En la captura se observa el archivo `backend-nestjs/schema.sql` (líneas 118 a 154) con la definición de las tablas `bookings` y `booking_events`.

Cada fila registra la cancha (`court_id`), el organizador (`organizer_id`), el canal (`APP`, `RECEPTION`, `PHONE` o `WHATSAPP`), el estado y los timestamps `starts_at` y `ends_at`. En las líneas 140 a 143 programamos `CONSTRAINT bookings_no_overlap EXCLUDE USING gist` usando la extensión `btree_gist`, lo que impide en el propio motor PostgreSQL que dos turnos activos (`HELD`, `PENDING_PAYMENT`, `CONFIRMED` o `BLOCKED`) se crucen en la misma cancha.

3.2 Lógica de reservas y pagos divididos en NestJS (`bookings.service.ts`)

El servicio `BookingsService` (en `src/bookings/bookings.service.ts`) ejecuta toda creación de reserva o pago dentro de una transacción SQL mediante `this.db.withTransaction(...)`.

Como muestra la captura del código en las líneas 91 a 128, cuando la reserva se crea en modo `SPLIT`, el servicio inserta el registro en `split_payments` con el token único `share_token` y vencimiento de 30 minutos, procesa el cobro del primer cupo con `this.culqi.createCharge(...)`, guarda el abono en `payment_shares` y registra el evento en `booking_events`.

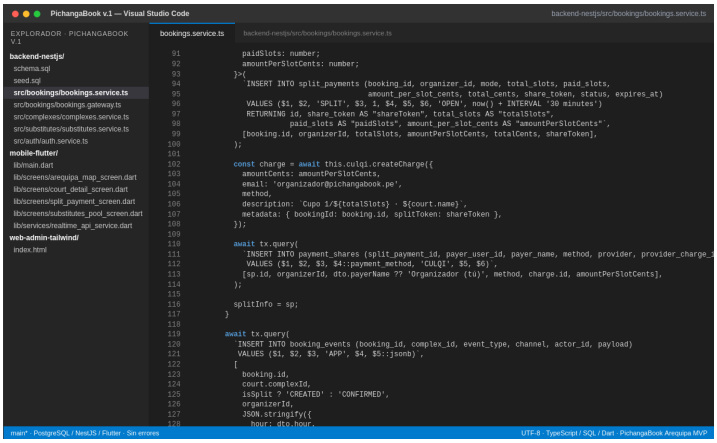


Figura 4: Transacción de reserva y creación de pago dividido en `bookings.service.ts`.

3.3 Gateway de sincronización en tiempo real con Socket.io (`bookings.gateway.ts`)

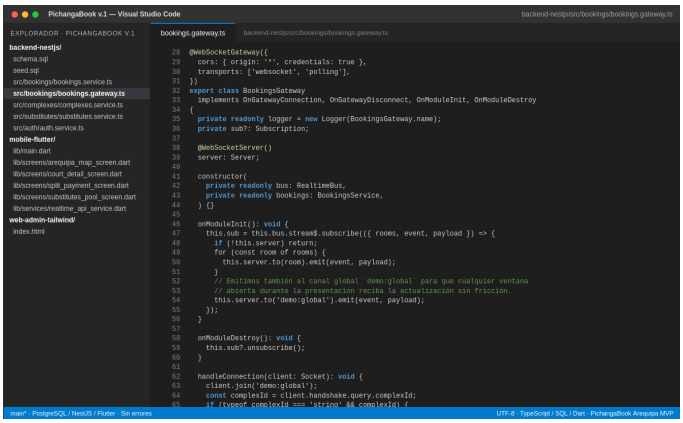


Figura 5: Implementación de `BookingsGateway` con Socket.io en NestJS.

Para que los bloqueos hechos en recepción y los abonos del pago dividido se reflejen al instante sin recargar la pantalla, implementamos la clase `BookingsGateway` en `src/bookings/bookings.gateway.ts` con `@nestjs/websockets` y **Socket.io**.

En las líneas 46 a 65 se observa cómo `onModuleInit()` se suscribe al bus reactivo `RealtimeBus` (RxJS) y difunde los eventos `slot:updated`, `split:updated` y `feed:event` hacia la sala del complejo (`complex:<id>`) y al canal `demo:global`.

3.4 Código del Frontend móvil en Flutter (`arequipa_map_screen.dart`)

En la carpeta `/mobile-flutter/lib` implementamos las pantallas móviles en **Dart / Flutter** (`arequipa_map_screen.dart`, `court_detail_screen.dart`, `split_payment_screen.dart` y `substitutes_pool_screen.dart`) conectadas al cliente `RealtimeApiService`.

La captura muestra las primeras 35 líneas de `arequipa_map_screen.dart`, donde se define el `StatefulWidget` con el paquete `google_maps_flutter`, el centro del mapa en Arequipa (`LatLng(-16.3880, -71.5430)`) y el catálogo de distritos (`CAYMA`, `YANAHUARA`, `CERRO_COLORADO`, `JLBYR` y `SACHACA`).

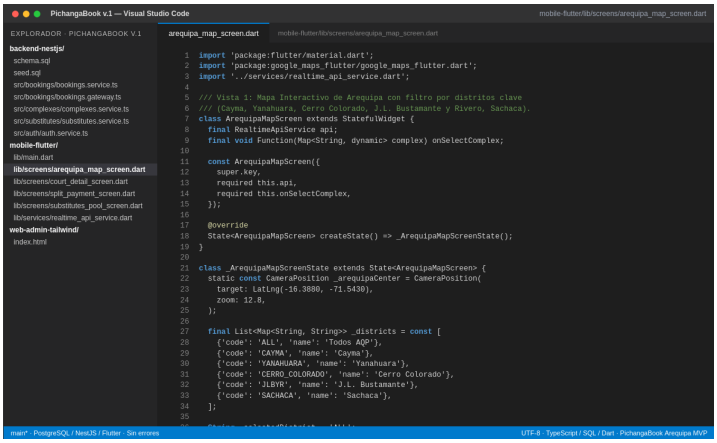


Figura 6: Código en Flutter (Dart) de la pantalla del mapa de Arequipa.

4 Evidencias Visuales del Frontend Conectado al Backend

4.1 Panel web para dueños de complejos deportivos

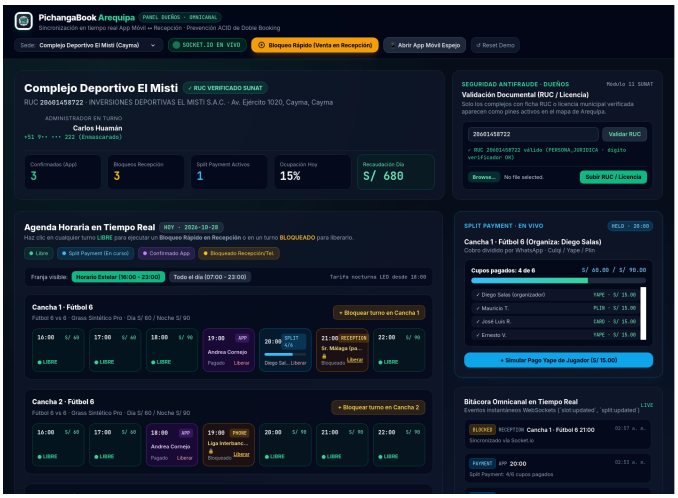


Figura 7: Panel web de administración para dueños de complejos corriendo en el navegador.

El panel web (`/web-admin-tailwind/index.html`) fue desarrollado con **HTML5**, **CSS3** y **Tailwind CSS**. Al iniciar consulta `GET /api/complexes` y `GET /api/owner/complexes/:id/dashboard` en el backend NestJS.

Muestra las métricas de ocupación del día, el formulario de validación de RUC mediante `GET /api/ruc/:ruc/validate` (algoritmo módulo 11 de SUNAT) y subida de licencia en `POST /api/complexes/:id/verification-document`, además de la parrilla horaria por cancha y la bitácora de eventos WebSocket.

4.2 Bloqueo rápido de turnos en recepción

Cuando ingresa un alquiler presencial en boletería o por llamada telefónica, el recepcionista abre este modal en el panel web, selecciona la cancha, la hora, el canal (`RECEPTION` o `PHONE`) y el nombre del cliente.

Al confirmar, el frontend envía `POST /api/bookings/reception-block` hacia `BookingsController`. El backend inserta el turno con estado `BLOCKED` en la tabla `bookings` y emite el evento `slot:updated` por Socket.io para actualizar al instante la parrilla de los jugadores.

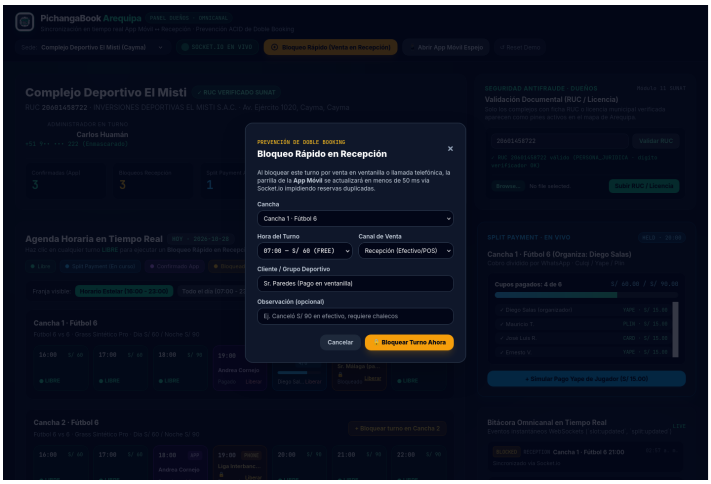


Figura 8: Modal de bloqueo rápido en recepción para ventas presenciales o telefónicas.

4.3 Mapa de complejos y filtros por distritos de Arequipa



Figura 9: Vista móvil del mapa de Arequipa y complejos filtrados por distrito.

La vista inicial de la aplicación móvil muestra el mapa de Arequipa con pines que indican la tarifa por hora de cada complejo deportivo y botones superiores para filtrar por distrito (obtenidos de la tabla `districts` mediante `GET /api/districts`).

Al elegir un distrito, la interfaz invoca `GET /api/complexes?district=...` en `ComplexesController`, que consulta en PostgreSQL los locales con `verification_status = 'VERIFIED'`, cruza su calificación desde la vista `v_complex_ratings` y calcula la distancia en kilómetros con la fórmula de Haversine.

4.4 Detalle de cancha y parrilla de disponibilidad en vivo

Al seleccionar un complejo se consulta `GET /api/complexes/:id` y `GET /api/courts/:id/availability` (servido por `AvailabilityService.courtGrid`), mostrando las canchas del local (Fútbol 6 y Fútbol 8) y los comentarios guardados en la tabla `reviews`.

La parrilla horaria diferencia los cuatro estados reales del turno: disponible en verde, reservado por app en morado (`CONFIRMED`), pago dividido en curso en azul (`HELD` mostrando el conteo `4/6`) y bloqueado por recepción en ámbar (`BLOCKED`).

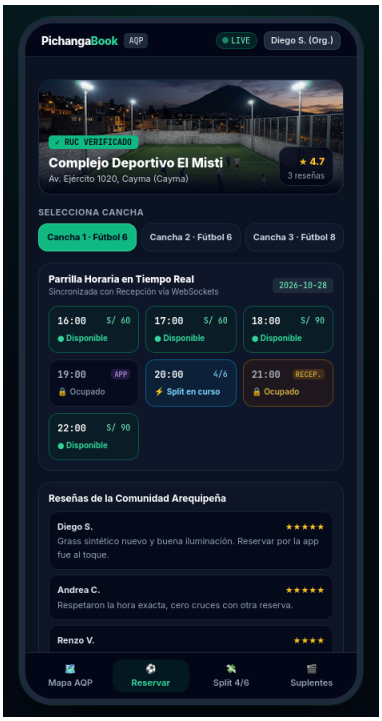


Figura 10: Parrilla horaria sincronizada y reseñas del complejo deportivo.

4.5 Pantalla de pagos divididos (Split Payment)



Figura 11: Contador de progreso de pago dividido (4 de 6 cupos pagados).

Esta pantalla consulta el estado del cobro fraccionado mediante `GET /api/split-payments/:token` (token `pk4de6demo`), mostrando la barra de progreso en vivo (*"Cupos pagados: 4 de 6 — S/ 60.00 / S/ 90.00"*) y el botón para compartir el link por WhatsApp.

Cuando un compañero abona su cuota de S/ 15.00 con Yape, Plin o tarjeta, la vista envía `POST /api/split-payments/:token/pay`. El método `paySplitSlot` bloquea la fila en `split_payments` con `FOR UPDATE`, inserta el pago en `payment_shares` y, al llegar a 6/6, cambia la reserva de `HELD` a `CONFIRMED`.

4.6 Bolsa de suplentes verificados con clips de video

La bolsa de suplentes consulta `GET /api/substitutes?position=...` en `SubstitutesController`, haciendo un `JOIN` en PostgreSQL entre las tablas `substitute_profiles`, `users` y `districts` para filtrar por posición (`ARQUERO`, `DEFENSA`, `MEDIOCAMPO` o `DELANTERO`).

Cada tarjeta muestra un reproductor de video con el clip técnico de 15 a 30 segundos (`clip_url` y `clip_duration_s`), el porcentaje de puntualidad (`punctuality_score`), los partidos jugados (`matches_played`) y el botón para convocarlo.



Figura 12: Tarjetas de suplentes verificados con reproductor de video de 15 a 30 s.

4.7 Seguridad: Verificación por SMS (OTP) y chat con número oculto

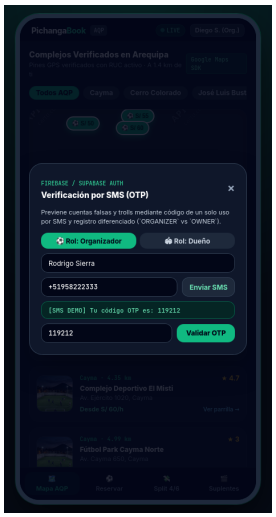


Figura 13: Registro por rol y validación por SMS (OTP).

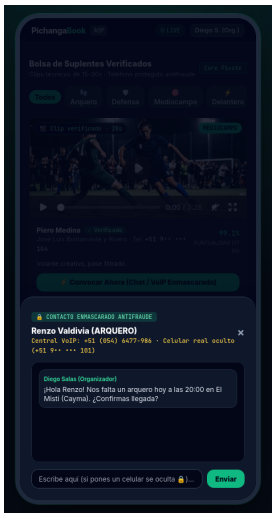


Figura 14: Chat interno con número celular enmascarado.

Para prevenir cuentas falsas, el modal de acceso (Figura 13) solicita un código OTP de 6 dígitos mediante `POST /api/auth/otp/request` y lo valida en `POST /api/auth/otp/verify` (gestionado por `OtpService` y `AuthService`), registrando al usuario en la tabla `users` con su rol (`ORGANIZER` u `OWNER`) y fecha de verificación `phone_verified_at`. Asimismo, al convocar a un suplente mediante `POST /api/substitutes/:id/invite` se crea una sesión en `contact_relays` con un `proxy_number` temporal (Figura 14); los mensajes enviados a `POST /api/relays/:id/messages` se guardan en `relay_messages` ocultando cualquier celular escrito en el texto y exponiendo solo la columna generada `phone_masked` (`+51 9•• ••• 101`).

5 Avance para la Semana Siguiente (6 al 12 de noviembre de 2026)

Para la entrega de la próxima semana nos enfocaremos en reemplazar los adaptadores de demostración por integraciones en modo de prueba real y completar los flujos pendientes entre el backend y la aplicación móvil:

Área	Tarea programada	Entregable / Evidencia para la próxima semana
Backend (Pagos)	Integración con el entorno Sandbox real de Culqi / Yape.	Conexión de `CulqiService` con llaves de prueba reales para procesar abonos en `POST /api/split-payments/:token/pay` y recibir webhooks en `/api/payments/webhooks/culqi`.
Backend y BD	Liberación automática de reservas vencidas (Cron Job).	Tarea programada en NestJS que libere los turnos en estado `HELD` y marque el `split_payment` como `EXPIRED` si no completan los cupos antes de `hold_expires_at`.
Backend y Cloud	Subida de clips cortos de suplentes a Cloudinary.	Flujo completo usando la firma de `POST /api/substitutes/clip-upload-signature` para subir videos reales de 15 a 30 segundos desde la app móvil.
Frontend Web	Historial de ingresos y exportación de reportes para dueños.	Vista adicional en el panel web (`/web-admin-tailwind`) con el resumen semanal de reservas por cancha y descarga en formato CSV.
Frontend Móvil	Compilación de APK en Flutter y pruebas en celulares Android.	Generación del archivo instalable APK desde `/mobile-flutter` conectado al backend desplegado en Render y pruebas con usuarios locales en Arequipa.

Tabla 1: Cronograma de tareas y entregables comprometidos para la semana del 6 al 12 de noviembre de 2026.